Final Project: Pick a Project

Computer Science - Week 10

Oct 3, 2026 - Version 0.0.2

Please make sure to your UD **email** AND **name** is below. Unless your UD emails are included in this table, then you will not earn any points for this assignment when it is graded! **Every person should submit their own individual worksheet**.

Talk about each of the questions with a friend or our LLM (or both), but **make sure the words you enter below are yours** (with the obvious exceptions notwithstanding). Brainstorming is best done in full sentences, and the main goal here is brainstorming. It's worth reviewing the information in this series of pages in the Drafter documentation to gain a better understanding of the process.

Role	Name	Email
Developer (You)	Jesse Marcozzi	jmrcozzi@udel.edu
Friend (optional)	LLM	Simple Expandable Project Ideas

Part 1: The Goal & Prior Implementations

1. **Identify the Goal:** What specific goal are you trying to address? Think about your passions, hobbies, and academic interests. The latter are an excellent candidate to focus on. Some majors have suggestions listed below: these were provided by faculty in those departments. Other majors could consider a focus tied to a big final assignment in one of their recent courses in their major.
    - **Biomedical Engineering** majors may consider one of the following:
        - If you have started (or completed) the Cell/Tissue Design final project in BMEG100, design an organoid, which involves combining a cell type, biomaterial, and testing method. Have the website include a description and diagram of your design concept, a testing method, a needs statement, and an impact statement relevant to their organoid. Also include a link to a paper that supports your idea and a paragraph describing how that paper supports your design concept.

        - Otherwise, create a website related to your BMEG100 Lab Report 1, *Cytoskeletal and Nuclear Staining Lab Report Guidelines*, linked here (University of Delaware account required to access).
    - **Chemistry** and **Chemical Engineering** majors may consider creating an automated chemistry assistant that takes balanced reaction equations, starting reactant masses, and a harvested product amount to determine the limiting reactant, theoretical yield, and percent yield based on principles from Chemistry 2e (OpenStax) — Chapter 4: Chemical Reactions and Stoichiometry. To avoid manually compiling atomic mass tables or writing custom string tokenizers, use the ChemPy library to parse molecular formulas into exact molecular weights and verify reaction stoichiometry against compound data from the NIH PubChem Database. The program outputs an organized breakdown of limiting reagent status, product yields, and remaining masses of unreacted species. Also use the heat of the reaction to determine if it is exothermic or endothermic, and compute the heat flow per mole of the product.
    - Majors in the **Electric & Computer Engineering** department may consider a steganography project that hides messages in images, and then retrieves those messages from uploaded images.
    - **Mechanical Engineering** majors may consider a project that somehow uses both the NumPy and SciPy Python libraries.

Personal spending tracker

2. **Prior Implementations:** Search the internet. What existing tools or websites currently attempt to address it?

There are a lot of tools that manage your budget available to install, however many of them are tied to your bank account (like Capital One). Some examples are Rocket money and Quicken. These are apps that all require an install, and some also require a subscription.

3. **Your Implementation:** Briefly describe the legitimate purpose or functionality of your site. How would you want this web application to be used?

My site can be used to quickly calculate money left over after spending, without requiring an install or subscription.

Part 2: The Input-Process-Output (IPO) Model

Think about the fundamental flow of your program.

1. **The Core Goal:** Every program has three main phases: taking input, processing data, and presenting output. In one sentence, what is the ultimate goal your program will accomplish for the user? Be specific and use CS1 concepts.

It will organize spent money through instances of a dataclass by category and produce leftover money and a sum of money spent on each category, with percentages through functions.

2. Input: What specific information will the user need to provide to your program? (e.g., a city name, a numerical budget, a list of hobbies).

Each transaction (money spent and category), total budget.

3. Processing: At a high level, how will the computer manipulate or evaluate that data? (e.g., calculate the average, filter out options that are too expensive, compare the input against a database).

It will calculate what percent of your budget you are spending on each category, as well as leftover money. It will also calculate your leftover money.

4. Output: What will the user get back? (e.g., a recommendation, a calculated safety factor, a formatted schedule).

It will return how much money you are spending on each category as well as the percent of your budget you spent on that category. It will also return your leftover money, or a warning if you exceeded your budget.

Part 3: Data Abstraction

Identify the "nouns" of your problem.

1. **Representing Reality:** Abstraction is the process of representing the real world inside a computer by ignoring unnecessary details. What are the core "things" (nouns) your program needs to track? (e.g., *If building a recipe app, the nouns are "Recipe" and "Ingredient"*).

Transaction, money, budget

2. **Identifying Properties:** For the most important "thing" you listed above, what 3 to 4 specific details do you actually need to solve your problem, and what details will you abstract away?
    - *Details I need:* (e.g., *Ingredient name, weight, cost*)
    - *Details I will ignore:* (e.g., *Ingredient color, brand, expiration date*)

Transaction: money spent, category, percent of budget

Part 4: Challenges, AI Strategy, & Goals

1. **Difficulties:** What parts of this project seem the most difficult or technically complex?

The hardest part of the project will be drafter syntax.

2. **Coding Strategy:** If you will be using an LLM to help code, how will you ensure you understand the generated code and avoid "hallucinations" or "shallow learning" (copying without understanding)? If you won't be using an LLM to help code, this project might take about 4 hours a week to complete, so list each of the days and times in which you work on the project.

I will use Monday, Tuesday, and Thursday afternoons from 4:30-7:00 to code. I will also likely use an LLM to code the drafter functions, generating the code in small portions and running after each section to debug and ensure it works as intended.

3. **Goals:** What is your ultimate goal for this project? (e.g., *Impress the graders with a creative UI, build something I can use in my other classes, learn Drafter deeply*).

I am hoping to apply dataclasses to represent the transactions through instances, and master using math on the dataclass values.

Part 5: Research & Brainstorming Conversation

Have a conversation with a Generative AI tool (like ChatGPT or Claude) OR a friend about your project idea. Do not ask for code. Instead, use the AI/friend as a sounding board to research your problem.

1. **The Initial Prompt/Question:** Ask the AI or your friend to help you brainstorm. (e.g., *"I want to build a simple web tool to help students track their study hours. What are 3 common problems students face when tracking their time, and what are 3 simple features a basic tool should have?"*). Paste your prompt below:

I want to make a spending tracker using drafter in python that takes the percentages of a budget and organises spending by category. *What are 3 common problems students face when tracking their budget, and what are 3 simple features a basic tool should have?*

2. **The Response:** Briefly summarize the best ideas the AI or your friend suggested. What existing solutions or prior apps did they mention?

The AI said that students may overspend, so I want to make sure the program accounts for overspending. It also recommended adding categorical percentage limits. It pointed out that students may also not know which category to put money in. It also recommended I create a budget progress display. It did not mention any existing apps or solutions similar to what I am making.

3. **The Pushback:** You must reply to the AI/friend to narrow the scope when it's over-ambitious. Pick the most complex feature they suggested and explain why it is too complicated for an introductory programming project. Ask them for a simpler alternative. *Paste your response here:*

A simpler alternative would be to let users enter their expenses and see how much they spend in total. The program could also allow users to assign each expense to a category, such as food, transportation, or entertainment, and calculate the total for each category. This is easier to implement because it only requires basic addition and grouping expenses.

4. **Reflection:** Based on this conversation, how has your project idea changed or become more focused?

Not really. The AI did not understand that categories will be entered by the user and the program will group transactions of the same category together. It also did not understand I will be using drafter in Thonny, and suggested I switch to Replit despite being the AI

provided for this project.

Part 6: Resource Planning

1. What resources will you use if you get stuck?

Provided LLM, office hours, drafter documentation.

2. If your LLM provides code that causes a Drafter or Bakery error, or if you run into a problem during deployment, what is your step-by-step debugging plan to resolve it?

I will show the LLM the error and ask it what happened, then ask it to fix the error. If I do this more than 3 times and it doesn't work, I will attempt to look for the error myself and possibly go to office hours if I am still stuck.

Part 7: LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

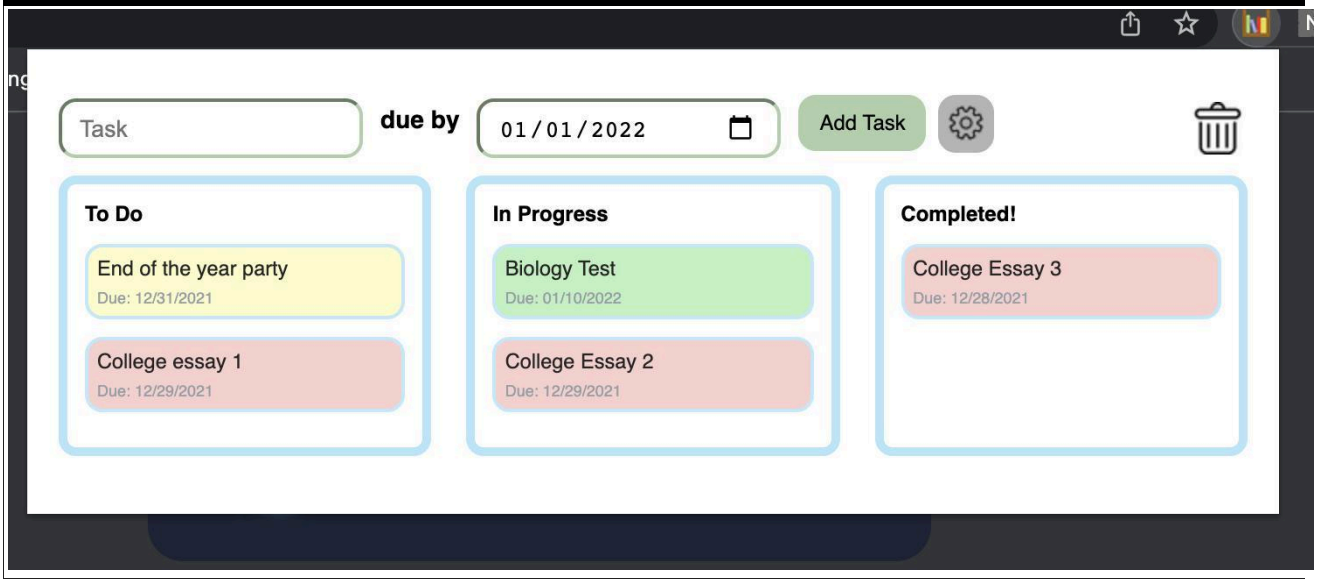
what are some simple project ideas that would be easy to augment if i need to accont for requirements later

For this project, I'd recommend choosing something with a very simple core version but several optional features you can add later. Since you're building with Drafter in an introductory CS1 course, the best ideas are ones that work with basic Python, lists, dictionaries, functions, and local storage rather than needing a backend or database.

Your project planning worksheet asks you to identify a problem, define inputs and outputs, model the important data, and explain how you'll keep the scope manageable.

CS1 Final Project Alpha F26.pdf So I'd pick a project where you can start small and still have room to grow if the requirements become more demanding.

6 simple project ideas that are easy to expand



1. Student task and assignment tracker

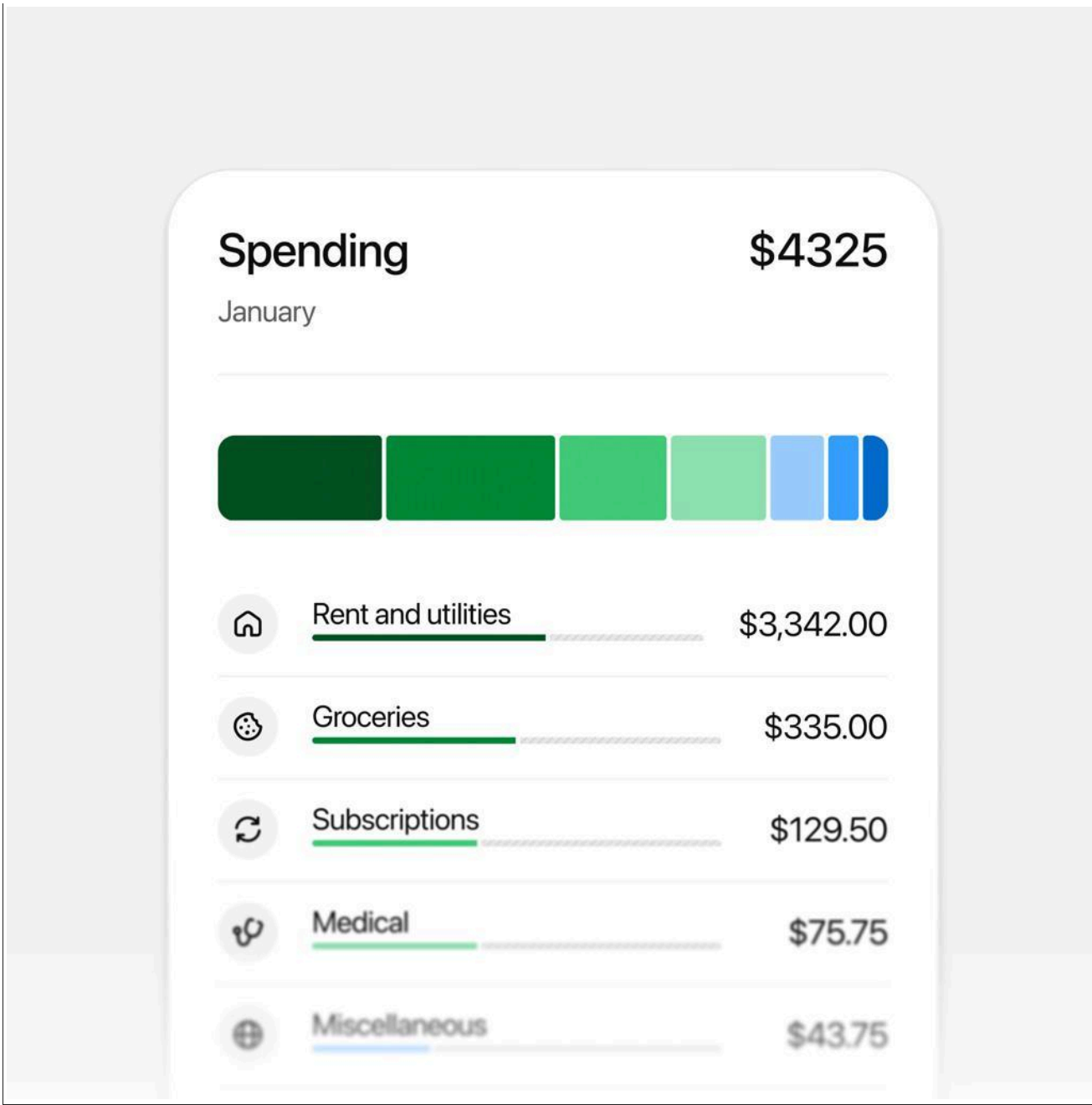
Easiest overall

Basic version: Add assignments with a name, course, due date, and completion status. Display them in a list.

Easy upgrades:

- Filter assignments by course or completion status.
- Sort assignments by due date.
- Show a progress bar for completed work.
- Save assignments using local storage.
- Add a calendar view or overdue warnings.

Skills: lists, dictionaries, dates, sorting, filtering.



2. Personal spending tracker

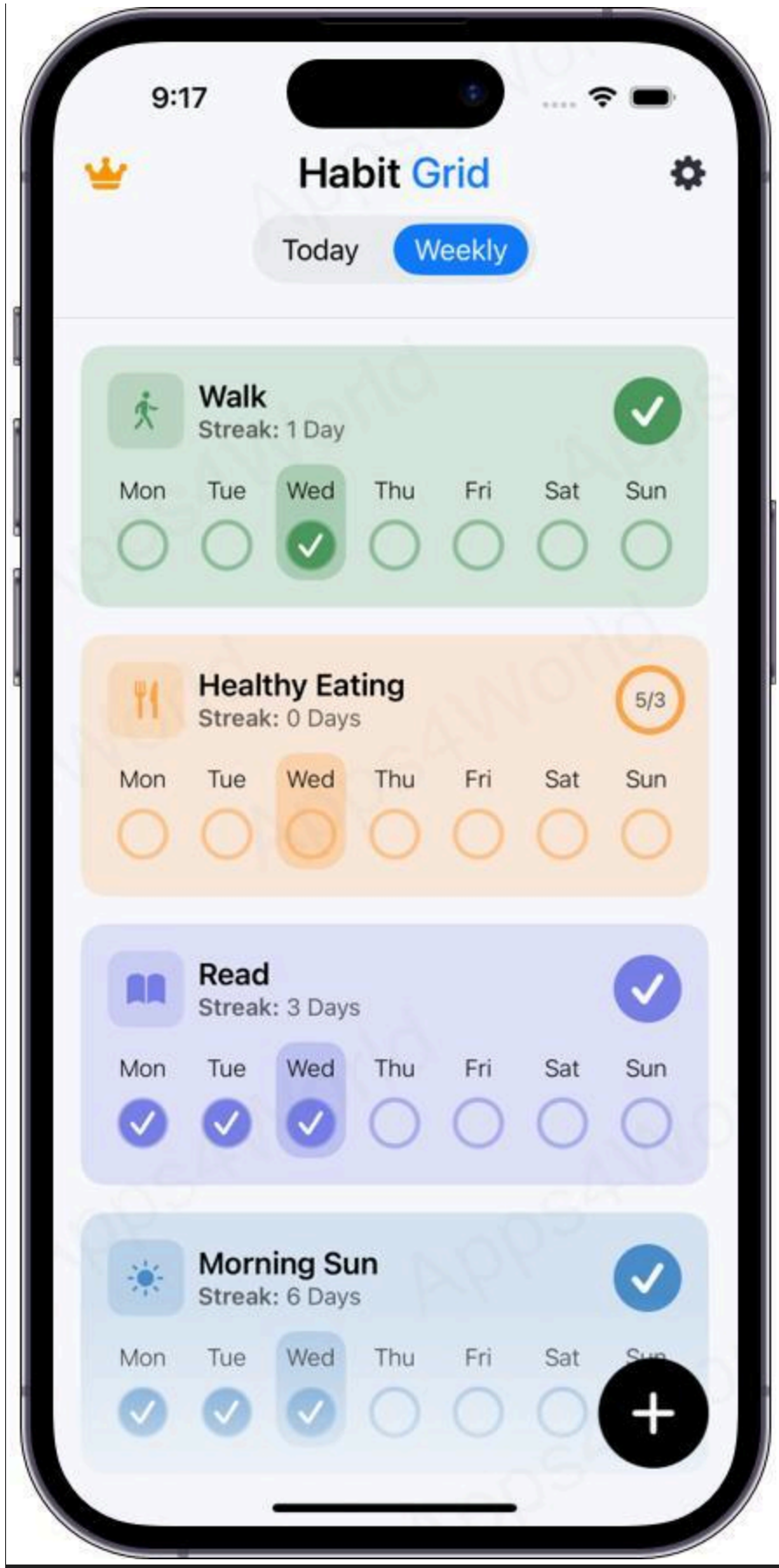
Basic version: Enter an expense, its cost, and a category. Calculate total spending.

Easy upgrades:

- Set a weekly or monthly budget.
- Display spending by category using a chart.

- Filter expenses by date.
- Show how much money remains in the budget.
- Export spending records to a file.

Skills: arithmetic, lists, totals, categories, charts.


9:17
Habit Grid
Today
Weekly
Walk
Streak: 1 Day
Mon Tue Wed Thu Fri Sat Sun
Healthy Eating
Streak: 0 Days
5/3
Mon Tue Wed Thu Fri Sat Sun
Read
Streak: 3 Days
Mon Tue Wed Thu Fri Sat Sun
Morning Sun
Streak: 6 Days
Mon Tue Wed Thu Fri Sat

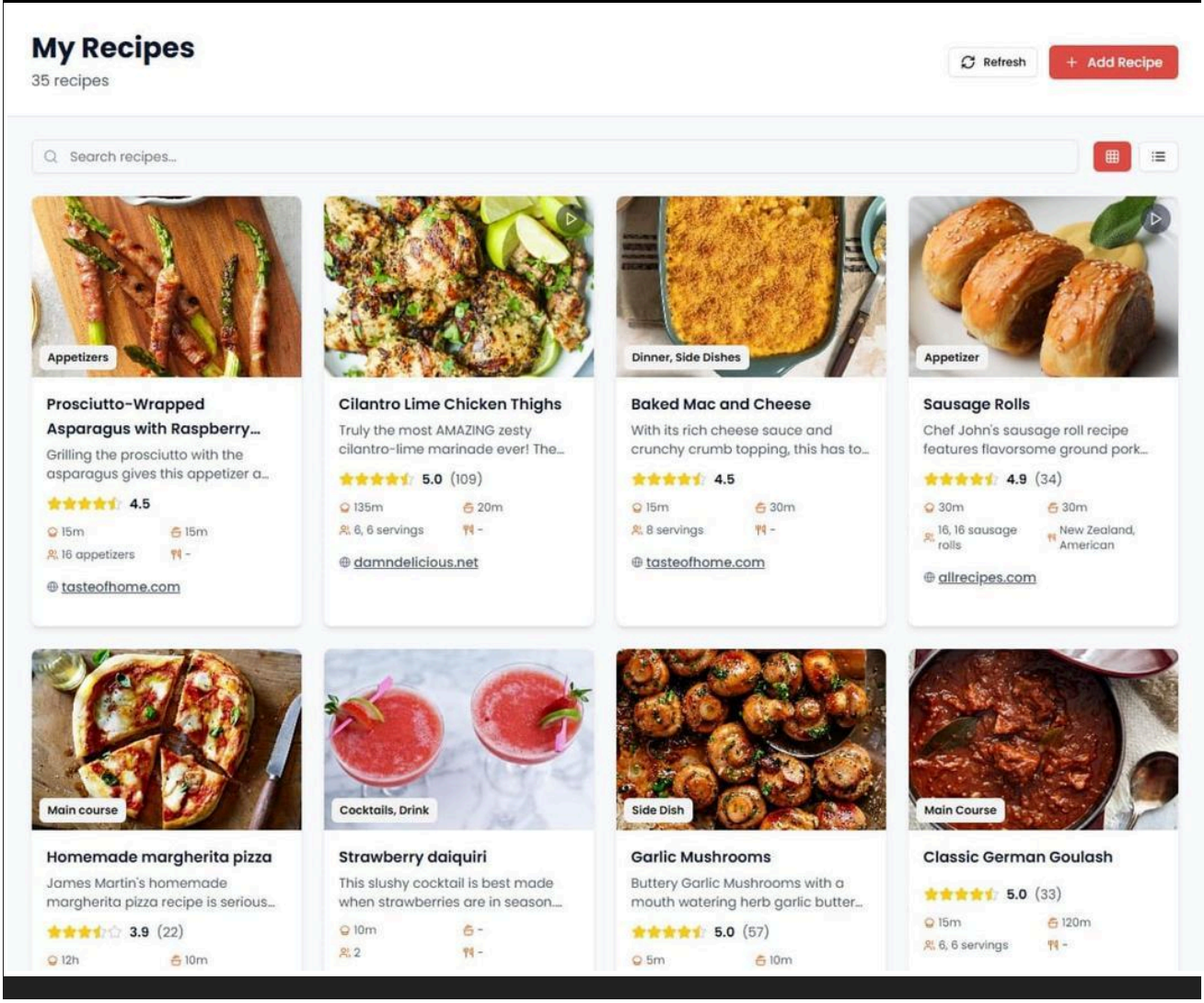
3. Habit tracker

Basic version: Let users create habits such as reading, exercising, or drinking water, then mark each habit complete.

Easy upgrades:

- Track daily or weekly completion.
- Calculate completion percentages.
- Display streaks.
- Show progress charts.
- Save habits and progress locally.

Skills: Boolean values, lists, dates, counters, data visualization.



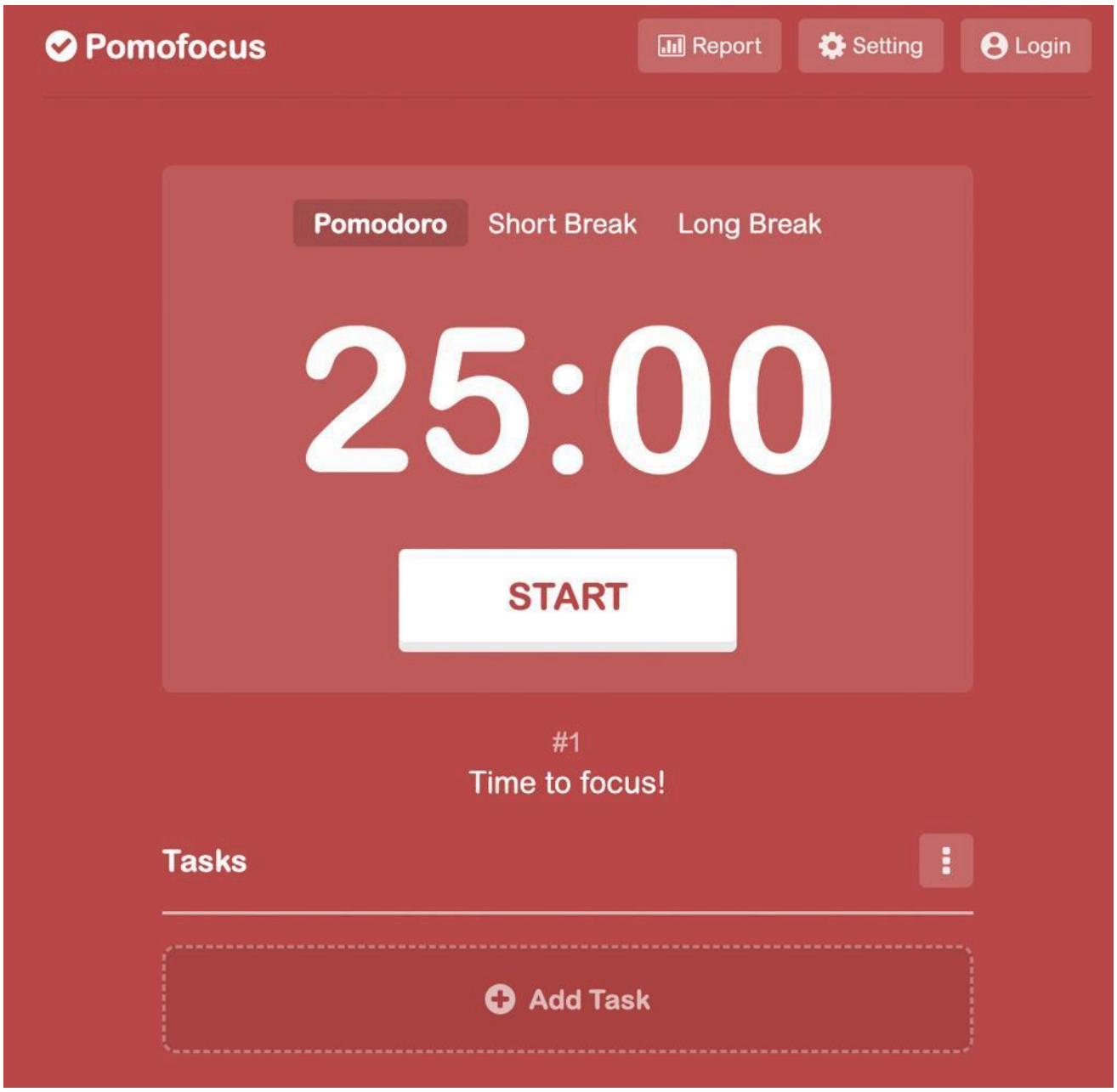
4. Recipe organizer and meal planner

Basic version: Add recipes with a name, ingredients, and preparation time. Display saved recipes.

Easy upgrades:

- Search by recipe name.
- Filter by ingredient or dietary preference.
- Scale ingredient quantities for different servings.
- Build a weekly meal plan.
- Generate a shopping list from selected recipes.

Skills: strings, dictionaries, searching, arithmetic, nested data.



5. Study session tracker

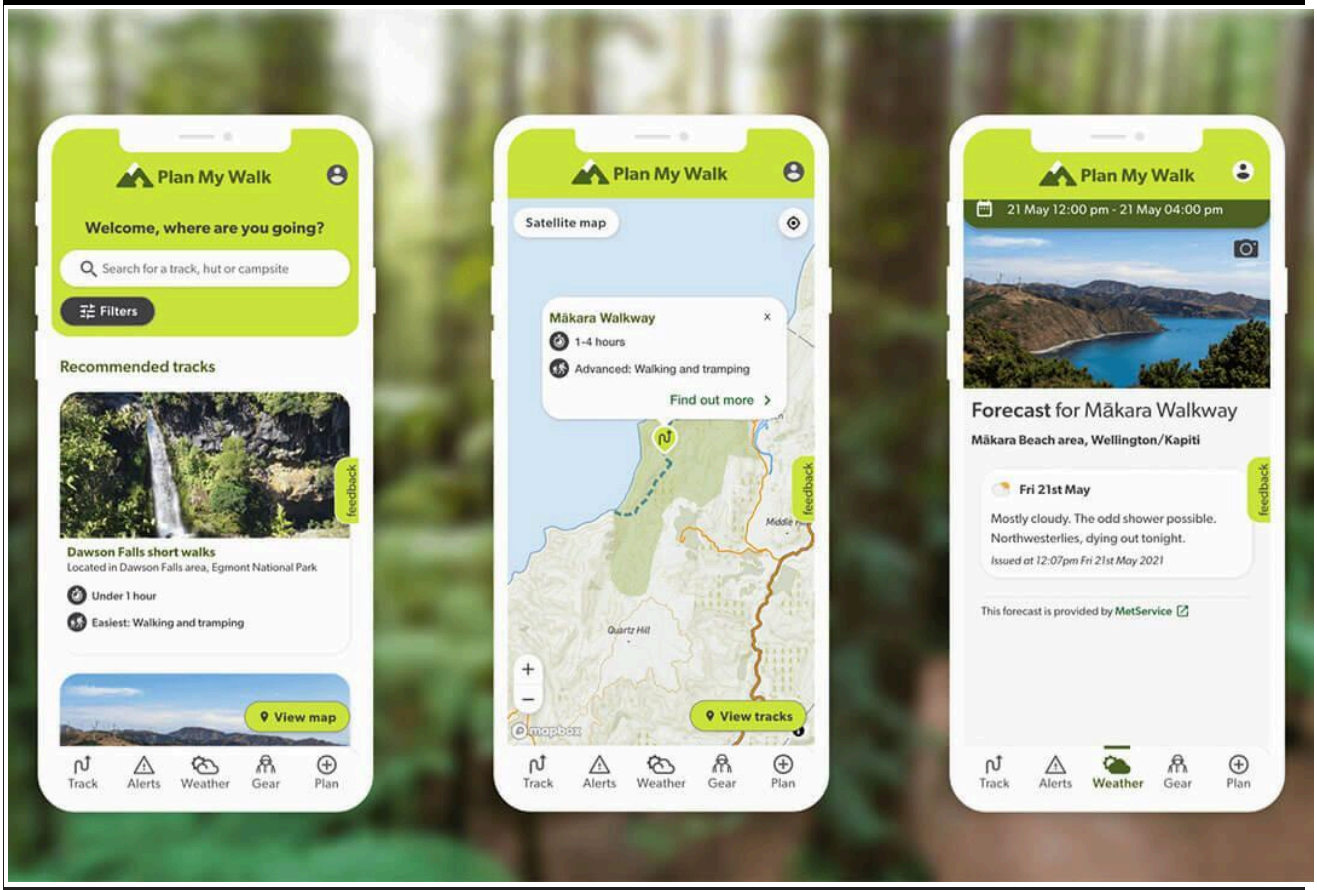
Basic version: Let users record a study subject and the number of minutes studied. Display total study time.

Easy upgrades:

- Add a study timer.
- Track time per course.

- Show daily or weekly statistics.
- Plot study time on a graph.
- Set study goals and display progress.

Skills: numeric calculations, timers, dates, charts, persistence.



6. Weekend activity planner

Basic version: Enter activities, locations, estimated costs, and preferred times. Display an itinerary.

Easy upgrades:

- Filter activities by budget or category.
- Calculate total estimated costs.
- Display locations on a map.
- Use geolocation or a supported weather API.
- Add a packing checklist.

Skills: lists, filtering, calculations, maps, optional web APIs.

Which project is easiest to adapt to new requirements?

Here's how I'd compare them for a three-week introductory programming project.

Project	Initial difficulty	Room to expand	Main risk
Assignment tracker	Very low	Excellent	Date handling
Spending tracker	Low	Excellent	More complicated charts
Habit tracker	Low	Excellent	Tracking dates consistently
Recipe organizer	Low	Very good	Managing nested data
Study tracker	Low	Excellent	Timer behavior
Activity planner	Medium	Very good	External APIs and maps

My top three would be the assignment tracker, spending tracker, and habit tracker. All three have clear input-process-output flows and simple data models, so you can add requirements without redesigning the entire application.

How to design your project so upgrades are easy

Whichever idea you choose, build it in layers.

1

Core functionality

Add records, display them, and perform one useful calculation or action.

This is your minimum viable project.

2

Data management

Add editing, deleting, sorting, filtering, and local storage.

These features are reusable across almost every project idea.

3

Visual polish and advanced features

Add progress bars, charts, timers, maps, or file export as needed.

Only add these once the core application works.

One important Drafter-specific consideration: because your application runs in the browser using Python and Pyodide, you should not depend on a custom backend or database. Local storage is a good way to persist a user's data on their own browser. External APIs can be possible, but may introduce network, permissions, or CORS issues, so I wouldn't make them essential to your basic version.

Let's narrow it down

If you know what kinds of requirements your instructor might add, we can choose an idea that accommodates them from the beginning.

1. What do you most want to optimize for?

Easy to add almost any new requirement

Looks impressive without much complexity

Something I'd actually use

Something a little more original

2. Which general category interests you?

Student productivity and school

Money and budgeting

Habits and personal goals

Food and organization

Games, hobbies, or something creative

Anything, as long as it's easy

3. How ambitious should the first version be?

As simple as possible

Simple, with one interesting feature

Basic at first, with room for advanced features

I want to make a spending tracker using drafter in python that takes the percentages of a

budget and organises spending by catagory. *What are 3 common problems students face when tracking their budget, and what are 3 simple features a basic tool should have?*

Since your project is a budget and spending tracker, you can adapt the question from time tracking to budgeting for Part 5 of your worksheet.

3 common problems students face with budgeting

1. Overspending in certain categories: Students may spend too much on food, entertainment, or shopping without realizing how quickly the costs add up.
2. Difficulty sticking to a budget: It can be hard to know how much money is left for each category after making several purchases.
3. Not knowing where their money goes: Without organizing expenses, students may struggle to identify which categories account for most of their spending.

3 simple features your basic tool should have

1. Budget percentage allocation: Let users enter a total budget and assign percentages to categories such as food, transportation, entertainment, and savings.
2. Expense entry and categorization: Allow users to enter a purchase amount and select its category. Automatically calculate the total spent in that category.
3. Budget progress display: Show the amount and percentage spent, the remaining budget, and whether a category has exceeded its limit.

Example of how your app could work

Suppose a student has a monthly budget of $1,000.

Category	Budget allocation	Budget amount
Food	30%	$300

Transportation	15%	$150
Entertainment	20%	$200
Savings	25%	$250
Other	10%	$100
Total	100%	$1,000

If the student spends $75 on food, your app would show that they have spent 25% of their food budget and have $225 remaining in that category.

My recommendation for your Drafter project

Start with just three things: set a budget, assign category percentages, and record expenses. Then, if you need more features later, you could add spending charts, alerts for overspending, and local storage to save the user's information between visits.

This gives you a manageable first version while leaving plenty of room to expand.

Creating a budget percentage allocation would be too difficult for me to implement. It would require me to already know what categories the user will enter before the program, which is impossible. It is also irrelevant to my goal of determining spending.

That's a reasonable concern, but there's one important distinction: your program doesn't need to know the categories in advance. You can let users create categories themselves. However, if your main goal is simply to track spending, you can also skip budget percentage allocation entirely.

For your project reflection, you could write:

Creating a budget percentage allocation feature would be too difficult for me to implement at this stage. I would need to handle user-defined categories, which could make the program more complicated than necessary. Also, allocating percentages is not essential to my main goal of tracking and organizing spending. A simpler alternative would be to let users enter expenses, assign each expense to a category, and automatically calculate the total spent in each category.

A simpler alternative for your Drafter app:

1. Let users enter an expense name, amount, and category.
2. Automatically group expenses by category and calculate each category's total.
3. Display total spending and a breakdown of where the money went.

Later, if you want to expand the project, you could add a bar chart comparing spending across categories or save expense records using local storage.

This approach is more manageable for a CS1 project because it focuses on basic Python concepts such as lists, dictionaries, loops, and arithmetic, without requiring complicated budget allocation logic.

For building your Python Draf ter project, here's a platform option.

give a simpler alternative

A simpler alternative would be to let users enter their expenses and see how much they spend in total. The program could also allow users to assign each expense to a category, such as food, transportation, or entertainment, and calculate the total for each category. This is easier to implement because it only requires basic addition and grouping expenses.

This might also interest you.

Part 8: Submit

In Google docs, with this document open, please click on *File* > *Download* > *Microsoft Word (.docx)*, and make sure you put that download somewhere you can find it again. Then, upload it to the Canvas assignment about selecting or brainstorming your final project.

When you're done with this assignment, review this page of the Drafter documentation to better prepare for the next step.